

System Design

Contents

The interview

1. [What the interview tests](#)
2. [A structure for the conversation](#)
3. [Clarifying requirements](#)
4. [Estimating scale](#)
5. [Numbers worth knowing](#)

Building blocks

6. [The edge](#)
7. [The application tier](#)
8. [Choosing a data store](#)
9. [Queues and event streams](#)
10. [Object storage, CDN and search](#)

Design levers

11. [Scaling reads](#)
12. [Scaling writes](#)
13. [Availability and redundancy](#)
14. [Choosing consistency per operation](#)

Running it

15. [Security as a design dimension](#)
16. [Designing for failure](#)
17. [Operating it](#)

Worked examples

18. [A URL shortener](#)
19. [A news feed](#)

Reference

20. [Interview questions](#)
21. [Common mistakes](#)

1. What the interview tests

A system design interview is not a test of whether we can recall an architecture. Interviewers know the candidate has read about news feeds. What they are watching is different:

- **Can we turn a vague prompt into concrete requirements?** "Design Twitter" is not a specification, and treating it as one is the most common way the conversation goes wrong.
- **Can we make a decision and defend the cost?** Naming a trade is worth more than picking the option the interviewer prefers.
- **Do we know what will break first?** A design that works at the stated scale and never mentions a bottleneck reads as untested.
- **Can we communicate?** The interviewer has to follow the design being built, in real time, without a diagram they can study afterwards.

There is rarely one right answer. Two candidates can reach different designs and both do well, because the evaluation is of the reasoning rather than the destination.

Two failure modes cause most poor interviews. The first is **naming components before establishing requirements**, which produces a design nobody can evaluate because the target was never set. The second is **describing an architecture with no trade-offs in it**, which sounds like a recital rather than a decision.

2. A structure for the conversation

A structure matters for a reason that has nothing to do with the design: it stops the panic, and it tells the interviewer where we are. A rough shape for forty-five minutes:

Time	Step	Output
5 min	Requirements	Functional list, non-functional targets, explicit scope cuts
5 min	Estimation	Requests per second, storage, bandwidth, and which one is hard
10 min	High-level design	Boxes, arrows, the path of one request and one write
15 min	Deep dive	One or two components, in detail, with alternatives
5 min	Failure and bottlenecks	What breaks first, what happens when it does
5 min	Wrap up	What is missing, what would come next

Say the plan out loud before starting. It costs twenty seconds, it frames everything after it, and it lets the interviewer redirect early if they want to spend the time elsewhere.

The deep dive is where the interview is actually won or lost. If the interviewer does not pick a component, offer the one with the most interesting trade in it, which is usually the data model, the write path, or whatever the estimation showed to be the hard dimension.

3. Clarifying requirements

Three lists, and the third is the one people skip.

Functional requirements are what the system does, written as operations rather than features. "A user can shorten a URL", "a user can retrieve the original URL", "an owner can see click counts". Keeping them as verbs makes the API fall out naturally.

Non-functional requirements are the numbers the design has to hit. Scale, latency, availability, consistency, durability, and how long data is kept. These are what make one design correct and another wrong, so they are worth asking for explicitly rather than assuming.

Out of scope is the list of things deliberately not being built. Saying "I am not going to design authentication, moderation or the mobile client" buys back time and shows judgment about what matters. An interviewer who wants one of them back will say so.

Questions worth asking, roughly in order of how much they change the design:

- **How consistent does this need to be?** This heavily influences the data model, the replication strategy and sometimes the store itself, so it is the highest-leverage question in the list.
- **Read-heavy or write-heavy, and roughly what ratio?** A hundred-to-one read ratio and a one-to-one ratio are different systems.
- **How many users, and how active?** The difference between a thousand and a hundred million is a different architecture, not a bigger one.
- **What is the latency target, and for which operation?** Usually only one or two operations are latency-sensitive.
- **One region or global?** Global brings replication lag and data residency with it.
- **What must never be lost?** Durability requirements differ per data type inside one system.

4. Estimating scale

The purpose of estimating is not arithmetic. It is to find **which dimension is hard**, because that dimension determines the design. A system with heavy traffic and tiny data is a different problem from one with light traffic and petabytes.

The method:

1. Start from users or requests per day, whichever the prompt gives.
2. Convert to per second.

- 3. Apply a peak multiplier, usually somewhere between two and ten.
- 4. Multiply item size by items per day by retention for storage.
- 5. Multiply item size by requests per second for bandwidth.

Rounding aggressively is correct here, and saying that out loud is better than pretending to precision. A day is 86,400 seconds; using 100,000 makes the division trivial and under-estimates by about fifteen percent, which is fine as long as it is stated.

A worked example:

```
100 million daily active users
× 10 requests each per day
= 1 billion requests per day

1 billion ÷ 100,000 seconds ≈ 10,000 requests per second average
× 3 for peak                ≈ 30,000 requests per second peak

Storage: 1 billion writes/day × 200 bytes = 200 GB/day
        × 365                      ≈ 73 TB/year
```

What that tells us is more useful than the numbers themselves. Thirty thousand requests per second is comfortably beyond one machine, so the application tier has to be horizontal. Seventy-three terabytes a year will not sit on one database node indefinitely, so partitioning or archival needs a mention. If the read-to-write ratio were a hundred to one, caching and read replicas become the first tools rather than sharding.

State the conclusion, not just the arithmetic: **"the hard part here is write volume"** or **"reads dominate, so this is a caching problem"** is what the estimate was for.

Peak is not capacity, which is the second conclusion worth drawing. A design whose peak load equals its maximum throughput has nothing left for a burst above the estimate, an instance failing, a deployment taking instances out of rotation, or load arriving unevenly across shards. So the useful output is the peak figure and how much room sits above it. Naming a fixed utilisation target is where this turns into folklore, since the right margin depends on how quickly capacity can be added and how bursty the traffic is.

5. Numbers worth knowing

These are order-of-magnitude figures that depend heavily on hardware, configuration and workload. They are for choosing between designs, not for capacity planning, and it is worth saying so when using them.

Latency, roughly:

Operation	Time
Main memory reference	~100 nanoseconds
Compress 1 KB	~1 microsecond

Operation	Time
Send 1 KB over a fast network	~10 microseconds
Solid state disk random read	~100 microseconds
Read 1 MB sequentially from memory	~10 microseconds
Read 1 MB sequentially from a solid state disk	~100 microseconds
Round trip inside one data centre	~0.5 milliseconds
Spinning disk seek	~10 milliseconds
Round trip between continents	~150 milliseconds

The two comparisons that matter most: **memory is roughly a thousand times faster than a solid state disk, and a cross-continent round trip is roughly a thousand times slower than one inside a data centre.** Those two ratios drive caching and geographic placement respectively.

Capacity, very roughly:

- One application server: thousands of requests per second for simple work, fewer if each request is expensive.
- One relational database node: thousands of writes per second on ordinary hardware, more with tuning and batching.
- One Redis instance: on the order of a hundred thousand operations per second.
- One Kafka broker: hundreds of thousands of messages per second, depending on size and durability settings.

Sizes:

Item	Size
<code>long</code> , <code>double</code>	8 bytes
UUID	16 bytes
A short text post	200 to 300 bytes
A typical web page	1 to 2 MB
A photograph	200 KB to 5 MB

Availability, in downtime per year:

Target	Downtime
99%	About 3.7 days
99.9%	About 8.8 hours

Target	Downtime
99.99%	About 53 minutes
99.999%	About 5 minutes

6. The edge

Everything in front of the application, in the order a request meets it.

DNS turns a name into an address. It can also do geographic routing, sending users to the nearest region, and simple round-robin distribution. Its limitation is the TTL: clients and resolvers cache answers, so a DNS change takes time to reach everyone and is a poor failover mechanism on its own.

Load balancers distribute requests across instances, and the layer they work at matters:

	Layer 4	Layer 7
Operates on	TCP connections	HTTP requests
Can route on	Address and port	Path, header, cookie, method
TLS	Can pass through, or terminate, depending on the implementation	Needs the plaintext request, so TLS is terminated at it or before it
Cost	Lower, connection-level	Higher, parses each request

Layer 7 is the usual choice for an HTTP service, because routing by path is what lets one address serve many services. Layer 4 suits raw throughput and non-HTTP protocols.

An API gateway sits at the boundary and handles what every service would otherwise repeat: authentication, rate limiting, routing, and request shaping. The API note covers those individually. The design point here is that it is one more component on the request path, so it needs the same redundancy as everything else.

A **CDN** sits in front of all of this for cacheable content, which the caching note covers.

7. The application tier

The single property that makes this tier easy is **statelessness**: no request depends on state held in one instance's memory, so any instance can serve any request. That is what allows adding instances behind a load balancer, replacing them during a deploy, and losing one without losing sessions.

State has to live somewhere, so the design has to say where:

State	Where it belongs
Session and identity	A signed token carried by the client, or a shared session store
Cached computation	A distributed cache, or a local one accepting divergence
Uploaded files	Object storage, with the reference in the database
Work in progress	A durable queue, so an application restart does not lose it

Sticky sessions, where a load balancer pins a user to one instance, are the option to mention and then argue against: they make deployment and scaling harder, and they reintroduce exactly the coupling statelessness removed.

Autoscaling should key off a signal that reflects real load. Processor use is the default and is often wrong for a service that spends its time waiting on a database; queue depth or request latency usually tracks the problem better.

8. Choosing a data store

The access pattern picks the store. The database note covers the internals; this is the selection decision.

Store	Suits	Example
Relational	Relationships, transactions, queries not known in advance	Postgres, MySQL
Document	Records read and written whole, varying shape	MongoDB
Key-value	Single-key access at very high volume	Redis, DynamoDB
Wide-column	Very high write volume, time-ordered data	Cassandra
Graph	Traversals several hops deep	Neo4j
Search index	Full text, ranking, faceting	Elasticsearch
Object storage	Large binary objects	S3
Time series	Metrics and measurements over time	Prometheus, InfluxDB

Two points worth making explicitly in an interview.

Relational is a reasonable default for most systems, and saying so is not a weak answer. It handles relationships, gives real transactions, and supports queries that were not anticipated when the schema was written. Moving away from it should be a decision with a reason attached.

One system can use several stores, which is called polyglot persistence. A single design might keep orders in Postgres, sessions in Redis, uploaded images in object storage and a search index in Elasticsearch. Each store is chosen for one access pattern, and the cost is that they now have to be kept in step, which is what change events and the outbox pattern are for.

9. Queues and event streams

Putting a queue between two components lets the producer stop waiting for the consumer, absorbs bursts rather than dropping them, allows failed work to be retried, and decouples the rate at which each side works.

Fanning one event out to several independent consumers is a different capability, and a plain competing-consumer queue does not provide it, since each message goes to exactly one consumer. That needs publish-subscribe, or an event log where each consumer group keeps its own position, which is the distinction below.

The distinction worth being precise about:

	Message queue	Event log
Example	SQS, RabbitMQ	Kafka, Pulsar
After consumption	The message is removed	The record remains until retention expires
Consumers	Usually compete for messages	Each keeps its own position
Replay	Not generally possible	Reading from an earlier offset
Ordering	Depends on the product, often limited to a queue or message group	Within a partition

A log suits several independent consumers and anything that may need reprocessing. A queue suits work that is done once and forgotten.

The design decision this creates is **what belongs in the request path**. A user waiting for a response should not wait for an email to send or a thumbnail to render. Placing a queue there turns a slow synchronous call into a fast acknowledgement plus asynchronous work, at the cost of the work no longer being complete when the response arrives, which the API contract has to reflect.

The distributed systems note covers ordering, delivery semantics and the outbox pattern that makes publishing reliable.

10. Object storage, CDN and search

Object storage holds large binary objects addressed by key. It is cheap, effectively unbounded and highly durable, and it is not a filesystem: there is no partial update and listing is not free. The pattern is to store the object and keep only its reference in the database, since putting megabytes into a relational row makes every query touching that table more expensive.

Uploads usually go direct from the client using a pre-signed URL, so bytes never pass through the application tier.

A **CDN** caches at the edge, near users. The caching note covers cache keys, invalidation and the failure modes. In a design, the point to make is which content is cacheable and for how long: static assets with content-hashed names can be cached indefinitely, while a personalised response must not enter a cache shared with other users unless the cache key and policy isolate them properly. A private cache, on the client or in the browser, is a different matter and is usually fine.

A **search index** is worth adding when the requirement is specialised: full text with ranking, faceting, fuzzy matching, or search at a scale the primary store is not the right tool for. Relational engines are not helpless here, and Postgres full-text search covers a good deal, so the argument for a separate index should be about what it does better rather than a claim that the database cannot. It is a derived store, never the source of truth, kept up to date from change events. That means it is eventually consistent with the database by construction, and a design should say what happens in the window where they disagree.

11. Scaling reads

Reads are the easier direction, and the order to try things matters because each step costs more than the last.

1. **Fix the query.** An index or a rewritten query is free compared with everything below it, and a cache in front of a query that needed an index just hides the cost.
2. **Cache.** The caching note covers the patterns and failure modes. This is usually the largest single improvement for a read-heavy system.
3. **Read replicas.** More read capacity without touching the write path. The cost is replication lag, so a user who writes and immediately reads can miss their own change, which is the read-your-writes problem.
4. **Denormalise.** Store the joined or aggregated result so reads do not compute it. The cost is keeping the copy correct.
5. **Precompute.** Build the answer ahead of time on write, which is the fan-out decision in section 19.

Stating this as an ordered list in an interview is itself worth something, because it shows the cheap options were considered before the expensive ones.

12. Scaling writes

Writes are harder, because they cannot be served from a copy.

1. **Batch.** Combining many small writes into fewer larger ones is often the biggest win available and costs only latency.
2. **Write asynchronously.** Accept the request, queue the work, acknowledge. This bounds the user-visible latency and moves the load into something that can be drained at its own rate.
3. **Split the write path.** Send different data to different stores, so one hot table is not the whole system's bottleneck.
4. **Shard.** Partition by a key so writes spread across nodes. The distributed systems note covers key choice, hot partitions and rebalancing, and the summary is that this is powerful and hard to reverse.

Two things to name before being asked. **Write amplification**: one logical write can become many physical ones once indexes, replicas and derived stores are counted, so a table with six indexes is doing seven writes. And **the single-leader ceiling**: in a single-leader setup, replication scales reads rather than write throughput, because every write still passes through the leader until the data is partitioned or the write architecture changes. Multi-leader and leaderless designs do distribute writes, at the cost of the conflict resolution the distributed systems note covers.

13. Availability and redundancy

Availability comes from having no single point of failure, which means redundancy at every layer that a request passes through, plus a way to detect and route around a failed component.

The arithmetic worth knowing, because it is counter-intuitive: **assuming failures are independent, components in series multiply**. Three components each at 99.9% give $0.999^3 \approx 99.7\%$, which is roughly a day of downtime a year rather than nine hours. Adding components to a request path lowers availability unless each one is itself redundant.

Redundancy in parallel goes the other way. Two independent components each at 99% give $1 - 0.01^2 = 99.99\%$, provided the failures really are independent, which shared power, shared network or a shared deployment can quietly break.

Scope	Protects against	Cost
Multiple instances	One process or machine failing	Small
Multiple availability zones	One data centre failing	Cross-zone latency and traffic charges
Multiple regions	One region failing, and distance to users	Replication lag, data residency, much more operational work

Multi-region is the one to adopt carefully. It brings the consistency questions from the distributed systems note into every write, and a common resolution is to keep writes in one region while serving reads globally.

14. Choosing consistency per operation

The distributed systems note covers the consistency models. The design skill is narrower: **decide per operation, not per system**.

Saying "the system is eventually consistent" is a weaker answer than walking through it:

Operation	Needs	Why
Deduct account balance	Transactional, strongly consistent	Two concurrent spends must not both succeed

Operation	Needs	Why
Reserve the last item in stock	Transactional, strongly consistent	Overselling is a real cost
Post appears in a follower's feed	Eventual	Seconds of delay is invisible to users
Like count on a post	Eventual	Approximate is acceptable, and it is very high volume
Search results include a new document	Eventual	The index is derived, and the window can be stated

The follow-up an interviewer often asks is what happens during the inconsistency window, and having an answer ready separates a memorised model from a design. For a like count it is that the number is briefly low. For stock it is that the design does not allow the window at all, because the check and the decrement happen in one transaction.

15. Security as a design dimension

Security belongs in the requirements from section 3, alongside scale and latency, rather than as a layer applied at the end. Four questions cover what a design-level discussion needs.

Who is the caller, and how do we know? Authentication answers who, and is usually done at the edge, with the gateway validating a token and passing on an identity. That is not the same as services blindly trusting whatever reaches them: the identity has to arrive in a form the service can verify, and anything reachable from outside the boundary has to authenticate for itself. The API note covers token handling and why revocation is the hard part.

What may this caller do, and to which data? Authorisation has to be checked against the specific object, not only the endpoint, which is the most common real vulnerability rather than a theoretical one. In a multi-tenant system every query needs scoping by tenant, and getting that wrong once leaks across customers.

How is abuse limited? Rate limits per caller, quotas, and a way to identify and stop one abusive client without affecting the rest.

Which data needs stronger protection? Encryption in transit is the baseline. Encryption at rest matters for regulated or sensitive data. Secrets belong in a secret manager rather than in configuration or images, and personal data usually arrives with retention and residency requirements attached.

The point that makes this a design dimension rather than a checklist: **security constraints change architecture**. Data residency can force regional partitioning. Tenant isolation can decide between one shared database and a database per customer. Both are much cheaper to raise while requirements are being set than to discover during the deep dive.

16. Designing for failure

The last five minutes of an interview usually go here, and having the answer prepared is worth more than the extra component it would otherwise be spent describing.

Bottleneck and failure are different questions, and separating them out loud is worth doing.

The **bottleneck** is whatever caps throughput while everything is healthy:

Application tier	30,000 requests/second	
Cache	100,000 requests/second	
Database	8,000 requests/second	← the bottleneck

Nothing is failing there. The database simply sets the ceiling, and adding application instances will not raise it.

Failure asks a different question: what happens when a component stops, or slows. A component can be the bottleneck and never fail, and one with plenty of headroom can still be the first thing to break.

What breaks first? That comes from the workload and the estimate rather than from a rule. The data store is often a candidate, because it holds state and is harder to scale than stateless compute, but depending on the system it could equally be an external dependency, a connection pool, a hot partition, a queue, or whatever has no timeout on it.

A checklist worth running in the interview:

- Where is the single point of failure? If any box appears once in the diagram, name what happens when it dies.
- What happens when the cache is empty, or gone entirely? The answer should be slower, not broken.
- What happens when a dependency is slow rather than down? Slow is harder, and it is what causes cascading failure.
- What happens when consumers fall behind? Bound the queue, apply backpressure, shed or coalesce work, or let lower-priority work expire. Unbounded is the option that turns overload into memory exhaustion.
- How is a bad deployment reversed, and how quickly?

Reversing a bad deployment needs tools rather than intent, and the list is short:

- **Rolling** replaces instances gradually. Two versions run at once, so every change has to be compatible in both directions while that lasts.
- **Blue-green** runs a second environment and switches traffic to it, so rollback is switching back, at the cost of running both.
- **Canary** sends a small share of traffic to the new version and watches the signals before continuing.
- **Feature flags** separate deploying code from enabling behaviour, so a bad feature is switched off without a rollback at all.

- **Expand then contract** for schema changes, because a rollback cannot undo a migration that dropped a column. Add the new shape, migrate, remove the old shape in a later release.

Graceful degradation is the answer worth naming: which features can be switched off while keeping the core working. A feed that cannot rank can still show posts in reverse order. A product page that cannot reach the recommendation service can render without recommendations. Deciding these in advance is design work, and doing it during an incident is not.

The API and distributed systems notes cover the mechanisms: timeouts, retries with backoff and jitter, circuit breakers, bulkheads, backpressure and load shedding. One of them is worth stating rather than only referencing, because it is the mechanism that turns a small outage into a long one: **retries have to be bounded and use exponential backoff with jitter**, since otherwise a struggling dependency receives more traffic exactly while it is trying to recover, and the retries become the outage.

17. Operating it

Three terms get used interchangeably and mean different things:

- **SLI, Service Level Indicator.** The thing measured. "The proportion of requests served in under 200 milliseconds."
- **SLO, Service Level Objective.** The internal target for that indicator. "99.9% of requests under 200 milliseconds, measured over 30 days."
- **SLA, Service Level Agreement.** A commitment to a customer, usually with a penalty attached. It is set looser than the SLO deliberately, so the team reacts before the contract is at risk.

A target means nothing until it names which indicator, over what window. "99.9% available" is an incomplete statement. These two are complete, and they are deliberately separate objectives rather than one:

- **Availability.** 99.9% of requests return a successful response, over a rolling 30 days.
- **Latency.** 99% of successful requests complete within 200 milliseconds, over a rolling 30 days.

Keeping them apart matters, because a service can meet one and miss the other, and the response to each is different: failing availability points at errors and capacity, while failing latency points at the tail.

The distributed systems note covers metrics, logs and traces and how they divide the work. What a design should add is **which signals would show this particular system in trouble**:

Signal	What it shows
Request rate, error rate, latency percentiles	Whether users are being served
Saturation: processor, memory, connections, threads	How much headroom is left before queuing starts
Queue depth and consumer lag	Whether asynchronous work is keeping up

Signal	What it shows
Cache hit ratio and miss latency	Whether the cache is earning its place
Replication lag	Whether replicas are safe to read from
Dependency latency and error rate	Whether an upstream or downstream service is becoming the constraint

Latency belongs in percentiles rather than averages. An average hides the tail, and the tail is what users notice, so p99 is the number worth putting in an objective. The caching note makes the same point from the other direction: a cache can halve the mean while barely moving p99, because the tail is still dominated by misses.

18. A URL shortener

Small enough to design completely, which is why it is asked so often.

Requirements. Shorten a long URL to a short code; redirect a short code to the original; optionally support custom aliases, expiry and click counts. Out of scope: user accounts, moderation.

Scale. Assume 10 million new URLs per day and a hundred-to-one read ratio.

Writes: $10 \text{ million} \div 100,000 \text{ s} \approx 100 \text{ per second}$
 Reads : $1 \text{ billion} \div 100,000 \text{ s} \approx 10,000 \text{ per second}$, say 30,000 at peak
 Storage: $10\text{M/day} \times 500 \text{ bytes} \approx 5 \text{ GB/day} \approx 1.8 \text{ TB/year}$

The conclusion to state: **this is a read-heavy system with modest write volume and modest storage.** So the interesting parts are the short code scheme and the read path, not sharding.

Generating the code. Four options, and the trade is the whole question:

Approach	Strength	Weakness
Hash the URL, take a prefix	Same URL gives the same code	Collisions need detection and resolution
Counter plus base62	No collisions, short codes	Sequential, so codes are guessable and volume leaks
Random, check for collision	Unguessable	A database check on every write
Pre-issued ranges per instance	No coordination per write	Gaps in the space when an instance dies

Base62 uses digits and both cases of the alphabet, so a seven-character code covers $62^7 \approx 3.5 \text{ trillion}$ values, comfortably more than needed.

```
private static final String ALPHABET =
    "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";

static String toBase62(long id) {
    StringBuilder out = new StringBuilder();
    do {
        out.append(ALPHABET.charAt((int) (id % 62)));
        id /= 62;
    } while (id > 0);
    return out.reverse().toString();
}
```

A reasonable answer combines two of them: a counter issued in ranges to each instance, so no coordination happens per write, with the counter value scrambled before encoding so the codes are not obviously sequential.

The read path. A redirect is a key lookup, which is what a cache is for. Client, load balancer, application, cache, and the database only on a miss. At a hundred-to-one read ratio the cache carries almost all of it.

Abuse deserves a sentence, since a service that turns any URL into a trusted-looking short link is attractive to whoever is sending phishing. Rate limiting per caller, scanning submitted URLs against a reputation service, and reserving obvious aliases are the usual answers, and section 15 covers why this belongs in the requirements rather than at the end.

One detail worth raising unprompted: the redirect status code is a trade. `301 Moved Permanently` is cached by browsers and proxies, so many later visits never reach the service. That is cheap, and it makes server-side click counting incomplete, since an unknown share of visits is answered by a cache we never see. `302 Found` sends every visit to the service, which costs traffic and preserves analytics. The right answer depends on whether analytics is a requirement, which is why section 3 asks.

19. A news feed

The classic design question, and the one where a single decision dominates.

Requirements. A user can post; a user can follow others; a user sees a feed of recent posts from the people they follow. Out of scope: ranking quality, ads, direct messages.

Scale. Assume 100 million daily active users, each reading their feed a few times a day and posting rarely, so reads dominate heavily.

The decision: fan-out on write against fan-out on read.

Fan-out on write, sometimes called push. When a user posts, the post identifier is written into a per-follower feed list at that moment.

- Reads are trivial: fetch one precomputed list.

- Writes are expensive in proportion to follower count.
- A user with ten million followers turns one post into ten million writes, which is the hot key problem from the caching note.

Fan-out on read, sometimes called pull. At read time, gather recent posts from everyone the user follows and merge them.

- Writes are trivial: one row.
- Reads are expensive, and get worse as the number of people followed grows.
- A user following five thousand accounts pays for that on every feed load.

A hybrid is the common production approach, because it avoids the worst case of either strategy, and saying so directly is the answer:

Ordinary user posts	→ fan out on write into follower feeds
Very-high-follower user	→ do not fan out
Feed read	→ precomputed list, merged with a pull from the small set of followed celebrities

The threshold is a tuning decision rather than a fixed number, and the reasoning is that the expensive path is used only where the cheap one fails.

Supporting details worth mentioning: the feed list per user is capped at a practical window, a few hundred entries in most products, with older entries fetched on demand. Where that cap sits is a product decision rather than a technical one, and worth naming as an assumption. Pagination is by cursor rather than offset, because a feed changes while it is being read, which the API note covers. And the feed list is a good fit for a key-value or wide-column store keyed by user, while posts, follows and profiles stay relational.

20. Interview questions

Each answer below is written to be said out loud as it stands. The last sentence always leads toward the next question rather than closing the topic.

"How would you approach a system design question you have not seen before?"

By spending the first few minutes not designing anything. I would establish the functional requirements as a short list of operations, then the non-functional numbers, which are scale, latency, availability and how consistent it has to be, and then say explicitly what I am leaving out so the time goes where it matters. After that I would do a rough estimate, because the point of it is to find which dimension is hard, since heavy traffic with tiny data and light traffic with petabytes are different problems. Only then would I draw the high-level path of one read and one write, and pick the component with the most interesting trade in it for the deep dive. I would also say that plan out loud at the start, because it lets the interviewer redirect me early if they want the time spent somewhere else.

"Walk me through estimating the scale of a system."

I start from whatever the prompt gives, usually daily active users or requests per day, and convert to per second, rounding a day to a hundred thousand seconds rather than eighty-six thousand four hundred so the arithmetic stays simple. I say that I am rounding, because it under-estimates by about fifteen percent and I would rather be explicit than falsely precise. Then I apply a peak multiplier, usually somewhere between two and ten depending on how bursty the traffic is, and separately work out storage as item size times items per day times retention. The output I care about is not the numbers, it is the conclusion: whether this is a read-heavy problem that caching solves, or a write-volume problem that needs partitioning, or a storage problem that needs archival. That conclusion then decides where I spend the rest of the interview.

"How do you choose a database for a design?"

By the access pattern rather than by preference. If the data has real relationships, needs transactions, and will be queried in ways nobody has anticipated yet, relational is the right default and I would say so rather than treating it as the unambitious answer. I would move away from it for a specific reason: a key-value store when access is always by a single key at very high volume, wide-column when write volume is very high and the data is time-ordered, a document store when records are read and written whole and the shape varies, a graph store when the queries traverse several hops. I would also expect to use more than one, since a single design might keep orders relational, sessions in Redis, images in object storage and a search index alongside, and then the real work is keeping the derived stores in step, which is what change events and the outbox pattern are for.

"Fan-out on write or fan-out on read for a feed?"

Both, split by follower count. A hybrid is common because it avoids the worst case of either strategy. Fan-out on write means a post is copied into each follower's feed list when it is published, so reads are just fetching one precomputed list, and the cost is that a user with ten million followers turns one post into ten million writes. Fan-out on read means gathering from everyone a user follows at read time, so writes are one row and reads get expensive for anyone following thousands of accounts. The hybrid pushes for ordinary users and skips the fan-out for very-high-follower accounts, then merges the precomputed list with a small pull from those few at read time. The threshold is a tuning decision, and the reason it works is that the expensive path only gets used where the cheap one breaks down.

"Your design has a cache. What happens when it fails?"

The service should get slower, not start failing, and if it cannot then the cache has become a dependency rather than an optimisation and that needs saying out loud. Two failures are worth separating: the cache being gone entirely, where every request goes to the database and the question is whether the database can survive the full load, and the cache being empty after a restart, which is the same load arriving all at once. For the first I would want the database to be sized for a degraded mode, or load shedding so a fraction of requests fail fast rather than all of them queuing. For the second, warming the cache before an instance takes traffic, or ramping traffic gradually, keeps the source from meeting full load with nothing cached. I would also check that expiry has jitter on it, because entries created together otherwise expire together and reproduce the same burst later.

"How do you make a system highly available?"

Redundancy at every layer a request passes through, and a way to detect and route around a failed component, since redundancy nobody switches to does not help. The arithmetic is worth stating because it is counter-intuitive: components in series multiply, so three components at three nines each give about 99.7 percent, which is a day of downtime a year rather than nine hours. That means adding a component to the request path lowers availability unless it is itself redundant. Parallel redundancy goes the other way, two independent components at 99 percent give four nines, but only if the failures really are independent, and shared power, shared network or a shared deployment quietly break that assumption. Then it is a question of scope: multiple instances, multiple availability zones, or multiple regions, with multi-region bringing the consistency and data residency questions into every write.

"What breaks first in your design?"

I would not assume a component in advance. I would use the workload and the estimate to find where the ceiling actually sits, and the database is often a candidate because it holds state and is harder to scale than stateless compute, but it could equally be a hot partition, a connection pool, a queue or an external dependency. Separately from the bottleneck, the thing most likely to take the service down is whichever remote call has no timeout, since that is how one slow dependency consumes every thread while everything else is healthy. The distinction I would draw is between a dependency being down and being slow, because slow is harder: a failed call returns quickly and can be handled, while a slow one holds resources, and that is what turns one component's problem into a cascading failure. So the things I would check are bounded queues so overload gets rejected rather than absorbed, timeouts on every remote call, circuit breakers so a failing dependency fails fast, and a decision made in advance about which features can be switched off while the core keeps working.

21. Common mistakes

- Naming components before establishing requirements, which produces a design nobody can evaluate because the target was never agreed.
- Describing an architecture with no trade-off in it, which reads as recall rather than judgment.
- Estimating without drawing a conclusion. The numbers exist to identify which dimension is hard.

- Treating estimates as precise. They are order-of-magnitude figures that depend on hardware and workload, and saying so is stronger than implying otherwise.
- Proposing sharding before indexing, caching and read replicas have been considered, when it is the most expensive and least reversible option.
- Saying "the system is eventually consistent" instead of deciding per operation, and having no answer for what happens during the window.
- Adding components to the request path without noticing that availability in series multiplies downward.
- Designing only the happy path, then having nothing to say when asked what breaks first.
- Making the cache load-bearing, so that losing it causes errors rather than slower responses.
- Assuming replicas scale writes. In a single-leader design every write still passes through the leader until the data is partitioned or the write architecture changes.
- Choosing a data store by familiarity and defending it afterwards, rather than by the access pattern.
- Designing for a scale nobody asked for. A system for a thousand users should not be presented as one for a hundred million.
- Going silent while thinking. The interviewer is evaluating reasoning they cannot see.

22. Quick reference

Step	Output
Requirements	Operations, non-functional numbers, explicit scope cuts
Estimation	Requests per second, storage, and which dimension is hard
High-level design	The path of one read and one write
Deep dive	One component, with the alternatives named
Failure	What breaks first, and the degraded behaviour

Rough number	Value
Seconds in a day	86,400, rounded to 100,000 for arithmetic
Memory reference	~100 nanoseconds
Solid state disk random read	~100 microseconds
Round trip in one data centre	~0.5 milliseconds
Round trip between continents	~150 milliseconds
99.9% availability	About 8.8 hours of downtime a year
99.99% availability	About 53 minutes
Three components at 99.9% in series	About 99.7%

Problem	First tools to consider
Reads too slow	Index the query, then cache, then read replicas
Writes too slow	Batch, then write asynchronously, then partition
One key far hotter than others	Local cache in front, or split the key
Storage growing without limit	Retention policy, archival, cold storage
One component is a single point of failure	Redundancy plus automatic detection and routing
A dependency is slow rather than down	Timeouts, circuit breaker, bulkhead, load shedding
Feed or timeline at scale	Fan-out on write, with a pull path for high-follower accounts
Derived store out of step	Change events from the source, usually via an outbox

Where the detail lives	Note
Transactions, isolation, anomalies	ACID transactions
Indexes, normalization, replication, sharding	Database fundamentals
REST, versioning, auth, rate limiting, idempotency	API and backend development
CAP, consensus, delivery semantics, tracing	Distributed systems
Cache patterns, eviction, stampede, CDN	Caching